

Estados y eventos



Manejo de eventos y estados en REACT

Manejo de eventos en react

- Los eventos en React son muy similares a los eventos del DOM estándar, pero con algunas diferencias importantes.
- Nombres de Eventos: Se usa camelCase (ej. onClick, onChange, onSubmit) en lugar de minúsculas (ej. onclick).
- Manejadores de Eventos: Se les pasa una función como valor, no una cadena de texto.
- React crea un "SyntheticEvent" que es un envoltorio compatible con diferentes navegadores

```
1  function BotonSimple() {
2    function handleClick() {
3      alert('¡Botón clickeado!');
4    }
5
6    return (
7      <button onClick={handleClick}>
8        Haz Clic Aquí
9      </button>
10    );
11  }
```



Manejadores de Eventos con Argumentos

A menudo necesitamos pasar información adicional a nuestros manejadores de eventos.

Esto se logra usando una función de flecha (arrow function) directamente en el onClick (o cualquier otro evento).

La función de flecha se encarga de ejecutar nuestro manejador con los argumentos deseados.

```
1 function ItemLista({ nombre }) {  
2   function handleComprar(itemNombre) {  
3     alert(`Comprando: ${itemNombre}`);  
4   }  
5   return (  
6     <li>  
7       {nombre}  
8       <button onClick={() => handleComprar(nombre)}>Comprar</button>  
9     </li>  
10  );  
11 }
```



INTRODUCCIÓN AL ESTADO (STATE)

El Estado → La Memoria de un Componente

- El Estado (State) es una característica fundamental de React que permite a los componentes tener su propia "memoria" o datos que pueden cambiar con el tiempo.
- A diferencia de las props (que son datos pasados de padre a hijo y son inmutables), **el estado es gestionado internamente por el componente y es mutable.**
- Cuando el estado de un componente cambia, React automáticamente "re-renderiza" (actualiza) el componente en la interfaz de usuario para reflejar esos cambios.
- Piensen en el estado como las variables internas de un componente que determinan cómo se ve y cómo se comporta en un momento dado.



```
1 const [contador, SetContador] = useState(0);
```



El hook → useState

Gestionando el estado con useState

En componentes funcionales, usamos el Hook useState para añadir y gestionar el estado.

Un Hook es una función especial de React que te permite "engancharte" a funcionalidades de React (como el estado o el ciclo de vida) desde componentes funcionales.

```
1 import React, { useState } from 'react';  
2  
3 const [nombreDeEstado, setNombreDeEstado] = useState(valorInicial);
```

nombreDeEstado: La variable que contiene el valor actual del estado.

setNombreDeEstado: Una función que usaremos para actualizar el estado. ¡Siempre usa esta función para cambiar el estado!

valorInicial: El valor con el que el estado comienza (puede ser cualquier tipo de dato: número, cadena, booleano, objeto, array).



EJEMPLO PRÁCTICO: CONTADOR

useState en Acción: Un Contador

- Veamos cómo un simple contador utiliza el estado para mantener y actualizar su número.
- Cada vez que se hace clic en el botón, la función setContador actualiza el estado contador, lo que provoca que el componente se re-renderice y muestre el nuevo número.

```
1  function Contador() {
2    const [cuenta, setCuenta] = useState(0); // Estado inicial en 0
3
4    const handleClick = () => {
5      setCuenta(cuenta + 1); // Incrementa la cuenta
6    };
7    return (
8      <div>
9        <p>Has clickeado {cuenta} veces</p>
10       <button onClick={handleClick}>
11         Incrementar
12       </button>
13     </div>
14   );
15 }
```



ACTUALIZANDO ESTADO BASADO EN EL ESTADO ANTERIOR

Actualizaciones Seguras del Estado

A veces, las actualizaciones de estado en React pueden ser asíncronas para optimizar el rendimiento.

Si tu nueva actualización de estado depende del valor anterior del estado (como en un contador), es más seguro pasar una función a la función set del estado.

Esta función recibe el estado anterior como argumento y devuelve el nuevo estado.

```
1  setCuenta(prevCuenta => prevCuenta + 1);
```



EVENTOS QUE MODIFICAN EL ESTADO

Eventos + Estado: El Corazón de la Interactividad

La combinación de eventos y estado es lo que hace que las aplicaciones de React sean dinámicas e interactivas.

Un evento (como un clic en un botón o un cambio en un input) dispara una función.

Esa función, a su vez, llama a la función set del estado (`setVariableDeEstado`).

Cuando el estado se actualiza, React re-renderiza el componente y la interfaz de usuario se actualiza automáticamente para reflejar el nuevo estado.

Flujo:

1. El usuario interactúa (dispara un evento).
2. El manejador de eventos se ejecuta.
3. El manejador actualiza el estado usando `setAlgo()`.
4. React detecta el cambio de estado.
5. React re-renderiza el componente con el nuevo estado.
6. La UI se actualiza.



COMPONENTES CONTROLADOS: FORMULARIOS

En HTML, los elementos de formulario (input, textarea, select) suelen gestionar su propio estado interno.

En React, es una buena práctica usar componentes controlados. Esto significa que el estado del formulario es gestionado por React.

El valor del input se conecta a una variable de estado (value={miEstado}).

Cada vez que el valor del input cambia, un manejador de eventos (onChange) actualiza esa variable de estado.

Esto nos da control total sobre los datos del formulario y facilita la validación y el manejo de los envíos.

Ejemplo Básico de Input Controlado:

```
1 function FormularioInput() {
2   const [valorInput, setValorInput] = useState('');
3
4   const handleChange = (e) => {
5     setValorInput(e.target.value); // e.target.value contiene el nuevo valor del input
6   };
7
8   return (
9     <div>
10      <input type="text" value={valorInput} onChange={handleChange} />
11      <p>Escribiendo: {valorInput}</p>
12    </div>
13  );
14 }
```

MÚLTIPLES ESTADOS EN UN COMPONENTE

Un componente puede tener múltiples variables de estado, cada una gestionada por su propia llamada a `useState`.

Cada `useState` es independiente.

`e` → es simplemente el nombre del parámetro que recibe información sobre el evento. No tiene que llamarse `e`, puedes usar cualquier nombre:

- `onChange={(e) => setEdad(parseInt(e.target.value))}`
- `onChange={(evento) => setEdad(parseInt(evento.target.value))}`

`target` → es una propiedad del objeto evento que apunta al elemento HTML que causó el evento.

1. Usuario escribe "25" en el input
2. React detecta el cambio y crea un objeto evento
3. React llama tu función `onChange` pasándole ese objeto
4. Tu función recibe el objeto en el parámetro `e`
5. Accedes al valor con `e.target.value`

```
function FormularioSimple() {
  const [nombre, setNombre] = useState('');
  const [edad, setEdad] = useState(0);

  return (
    <div>
      <input
        type="text"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
        placeholder="Tu nombre"
      />
      <p>Nombre: {nombre}</p>

      <input
        type="number"
        value={edad}
        onChange={(e) => setEdad(parseInt(e.target.value))}
        placeholder="Tu edad"
      />
      <p>Edad: {edad}</p>
      <button>Enviar</button>
    </div>
  );
}
```



EJERCICIO PRÁCTICO: UN FORMULARIO INTERACTIVO

- Crea un nuevo componente llamado `FormularioSaludoPersonalizado`.
- Este componente debe incluir:
- Un input de texto (`<input type="text" ...>`) para que el usuario introduzca su nombre.
- Un botón (`<button>`) con el texto "Saludar".
- Un párrafo vacío (`<p>`) debajo del botón.
- Usa el Hook `useState` para gestionar el nombre que el usuario introduce en el input. Asegúrate de que el input sea un componente controlado.
- Cuando el usuario haga clic en el botón "Saludar", el texto del párrafo debe actualizarse para mostrar: "¡Hola, [nombre introducido por el usuario]!"